as https://github.com/egulias/EmailValidator, and URLs need to be validated with support for IDN (Internationalized Domain Name), punycode (ASCII representation of Unicode), and IPv6 addresses – https://github.com/Fleshgrinder/php-url-validator is one such library.

Remember: *trust no one*.

Summary

In this, the final chapter of the book, we have delved into the very reason for PHP's existence: to create web content. We covered the fundamentals of how web clients and servers communicate with each other with the HTTP protocol by examining in detail the special messages that are exchanged between them. HTTP requests are composed of a request line, which tells the server what the request method is, the resource the client would like to access, and the version of the protocol to use. Header lines provide more detail concerning what the client's capabilities are, as well as those all-important cookies. We then looked at sessions, which are used to manage state in what is otherwise a stateless protocol.

With the nuts and bolts of HTTP in place, we moved onto the main event: HTML. Beginning with the fundamentals of HTML structure and syntax, we covered the parsing of HTML into a Document Object Model (DOM) data structure and navigating/modifying it with `Dom\HTMLDocument` et al. before progressing to generating markup. My approach to HTML generation was to delegate the task to a library – the popular Twig templating library, to be precise. Other PHP books might prefer to focus on using PHP itself as the templating language, for that is how PHP started life. However, given the enormous increase in the scope of the language, I feel that today's PHP offers too much in the context of generating web content (a "view" in MVC terminology). Twig or similar alternatives are de facto standard tools in modern PHP development, and knowing how to use templates will serve you better than knowing how to do it "the old-fashioned way."

Moving on from Twig, we looked under the bonnet at something Twig does by default: filtering data. The reasons for why this is necessary were repeating themes throughout the chapter: trust nobody on the internet. All incoming data is to be viewed as a potential source of malicious effects, even if you or your organization own the source. Rounding out the section on `htmlentities()`, etc., was a relatively light-hearted look at PHP's `filter` extension: useful within its limits but with some interesting foibles.